

Web Bluetooth 기반 무설치 교육용 자율 로버 플랫폼

ARES의 설계 및 구현

Design and Implementation of ARES, an Install-Free

Web Bluetooth Educational Autonomous Rover Platform

저자명*

(주)코리아사이언스 / 동명대학교 게임학부

2026년 5월

초록

피지컬 컴퓨팅 교육에서 교육용 로봇 플랫폼은 학습자가 알고리즘을 하드웨어로 직접 구현하며 컴퓨팅 사고력을 체득하게 하는 효과적인 도구이다. 그러나 기존 다수의 플랫폼은 전용 애플리케이션이나 PC용 연결 프로그램의 설치를 전제로 하여, 크롬북(Chromebook)·웨일북 등 경량 운영체제 기반 기기가 보급된 교육 현장에서 접근성과 수업 준비 효율을 저해한다. 본 연구는 별도의 설치 과정 없이 웹 표준 브라우저만으로 로버를 제어하는 Web Bluetooth 기반 교육 플랫폼 **ARES**(Autonomous Rover Exploration System)를 설계·구현한다. 구현의 핵심 난점은 사양이 제각각인 저가 HM-10/BT05 클론 BLE 모듈과 RAM이 제한된 MicroPython 런타임이라는 척박한 토대 위에서 다양한 길이의 명령과 센서 응답을 양방향으로 신뢰성 있게 주고받는 데 있다. 본 논문은 이를 네 가지 설계 과제로 정식화하고, 각 과제에서 순진한 접근이 실패하는 이유와 채택한 해법·근거를 기술한다. 구체적으로 (1) BLE 20바이트 전송 제약을 극복하는 개행 종결 체크 스트리밍, (2) 클론 모듈의 비결정성에 대응하는 전송 모드 선택과 페이싱, (3) 비동기 응답 경합을 제거하는 프로미스 선선평정과 fire-and-forget 분리, (4) 제한 자원 위에서의 체크 재조립과 접두사 기반 디스패치를 다룬다. 아울러 12차시 48미션 화성 탐사 교육과정을 제시하고, 기능 검증 결과와 위 설계 해법의 효과를 정량 검증하기 위한 측정 프로토콜을 제시한다.

주제어: Web Bluetooth, BLE, 블록 코딩, Google Blockly, 라즈베리파이 피코, 피지컬 컴퓨팅, 무설치 교육 플랫폼

1 서론

지능정보사회의 도래와 함께 소프트웨어·인공지능 소양 교육의 저변이 확대되면서, 추상적 개념을 물리적 결과로 환원해 보여 주는 피지컬 컴퓨팅 교육의 중요성이 커지고 있다. 하드웨어를 직접 제어하며 알고리즘을 설계하는 과정은 학습자가 순차·반복·조건 등의 컴퓨팅 사고 요소를 시각적이고 직관적으로 체득하도록 돕는다.

그러나 기존의 다수 교육용 로봇 플랫폼은 전용 모바일 애플리케이션이나 PC용 드라이버·연결 프로그램의 설치를 필수로 요구하는 구조적 한계를 지닌다. 이러한 방식은 학교·공공 교육 현장에 널리 보급된 크롬북, 웨일북 등 경량 운영체제 기반 기기에서의 접근성을 저해하며, 임의의 소프트웨어 설치가 제한되는 관리형 단말 환경에서는 도입 자체가 어렵다. 또한 수업 시작 전 기기마다 프로그램을 설치·설정하는 데 적지 않은 시간과 관리 비용이 소요되어 실제 학습 활동 시간을 잠식한다.

본 연구는 이러한 **설치 장벽** 문제를 해소하기 위해, 별도의 설치 과정 없이 웹 표준 브라우저에서 직접 로버를 제어하는 Web Bluetooth 기술 기반의 교육 플랫폼 **ARES**(Autonomous Rover Exploration System)를 제안한다. ARES는 초등학생 수준 학습자를 대상으로 하며, Google Blockly 기반 블록 코딩 인터페이스로 작성한 제어 명령을 브라우저가 BLE(Bluetooth Low Energy)를 통해 라즈베리파이 피코 기반 로버 “알비”로 직접 전송한다. 학습 내용은 화성 탐사라는 단일 서사 아래 12차시 48미션으로 구조화되어, 단계적 난이도로 블록 코딩 역량을 함양하도록 설계되었다.

*교신저자: young.min.kang@gmail.com

무설치라는 목표는 단순히 Web Bluetooth API를 호출하는 것만으로 달성되지 않는다. 교육 현장의 단가 제약 상 사용하게 되는 저가 HM-10/BT05 클론 BLE 모듈은 표준 사양과 달리 한 번에 보낼 수 있는 페이로드가 작고 connection interval과 지원 전송 모드가 제품마다 다르며, 메인 컨트롤러인 라즈베리파이 피코의 MicroPython 런타임은 RAM이 극히 제한적이다. 본 연구의 실질적 기여는 이러한 비이상적 토대 위에서 다양한 길이의 명령과 센서 응답을 **신뢰성 있게** 주고받는 통신 · 재조립 계층을 설계한 데 있다. 구체적 기여는 다음과 같다.

- **무설치 웹 제어 구조:** 중계 플러그인이나 로컬 백그라운드 프로그램 없이, 웹 표준 Web Bluetooth API만으로 BLE 하드웨어를 직접 제어하는 3계층 아키텍처를 설계하였다.
- **불안정 링크 대응 경량 통신 프로토콜:** BLE의 20바이트 전송 제약과 클론 모듈의 비결정성, 비동기 응답 경합, 제한 자원이라는 네 가지 설계 과제를 정식화하고, 개행 종결 청크 스트리밍 · 전송 모드 선택과 페이싱 · 프로미스 선선평과 fire-and-forget · 접두사 기반 CommandProcessor로 해결하였다.
- **시나리오 기반 교육과정:** 화성 탐사 서사를 중심으로 한 12차시 48미션 교육과정을 플랫폼에 내장하여, 하드웨어 구성 요소를 단계적으로 도입하는 학습 경로를 제공하였다.

본 논문의 구성은 다음과 같다. 2장에서 관련 연구를 검토하고, 3장에서 시스템 아키텍처를 기술한다. 4장에서 네 가지 설계 과제와 그 해결을 상세히 설명하고, 5장에서 기능 검증과 측정 프로토콜을 제시한다. 6장에서 결론과 향후 과제를 논의한다.

2 관련 연구

2.1 교육용 블록 코딩 플랫폼

스크래치(Scratch)[1], 엔트리(Entry)[2], Google Blockly[3] 등은 텍스트 문법 학습 부담을 제거한 시각적 블록 구성을 통해 프로그래밍 입문 장벽을 크게 낮춘 대표적 도구이다. 특히 Blockly는 블록 조립 결과를 JavaScript · Python 등으로 변환하는 코드 생성기를 제공하여, 웹 애플리케이션에 임베드되는 블록 에디터의 사실상 표준 라이브러리로 자리 잡았다.

그러나 이들 플랫폼이 실제 하드웨어 로봇과 연동될 때에는 공통적인 불편함이 있다. 웹 브라우저의 샌드박스 보안 모델은 기본적으로 임의의 물리 장치 접근을 허용하지 않으므로, 종래에는 시리얼 · 블루투스 포트를 중계하는 별도의 로컬 백그라운드 프로그램(connector/agent)이나 브라우저 확장을 설치해야 했다. 이 중계 계층은 설치 · 권한 · 버전 호환성 문제를 유발하며, 관리형 단말 환경에서 도입을 어렵게 만드는 주된 요인이다.

2.2 Web Bluetooth 기반 하드웨어 제어

Web Bluetooth API[4]는 브라우저가 중계 프로그램 없이 직접 BLE 기기와 GATT(Generic Attribute Profile) 통신을 수행할 수 있도록 하는 웹 표준 기술이다. 크로미움 계열 브라우저의 지원 확대로, 웹페이지 접속만으로 물리 장치를 제어할 수 있는 기술적 토대가 마련되었다. 다만 Web Bluetooth는 사용자 제스처 기반의 기기 선택, 보안 컨텍스트(HTTPS 또는 localhost) 요구, 그리고 BLE 링크 계층의 작은 전송 단위라는 제약을 수반한다. 더욱이 교육 현장에서 비용 문제로 채택되는 HM-10/BT05 호환 클론 모듈은 표준 동작이 보장되지 않아, 다양한 길이의 제어 명령과 센서 응답을 양방향으로 안정적으로 주고받기 위해서는 이러한 제약과 비결정성을 흡수하는 응용 계층 프로토콜 설계가 반드시 필요하다. 본 연구는 이 지점에서 경량 청크 스트리밍 프로토콜을 설계하여 교육용 로버 제어에 적용한다.

3 시스템 아키텍처

ARES는 사용자 편의성과 임베디드 처리 효율을 동시에 확보하기 위해 **웹 인터페이스 계층, 무선 통신 계층, 임베디드 제어 계층**의 3단 파이프라인으로 설계되었다(그림 1). 각 계층의 역할과 계층 간 인터페이스는 표 1과 같다.

표 1: ARES 3계층 파이프라인 구성

계층	구성 기술	역할
웹 인터페이스	HTML/JS, Google Blockly	블록 조립 → 제어 명령 문자열 생성, 센서 응답 시각화
무선 통신	Web Bluetooth (BLE GATT)	20바이트 청크 양방향 스트리밍, 응답 대기/타임아웃 관리
임베디드 제어	Raspberry Pi Pico (MicroPython)	청크 재조립, 접두사 기반 명령 디스패치, 하드웨어 구동 및 센서 측정

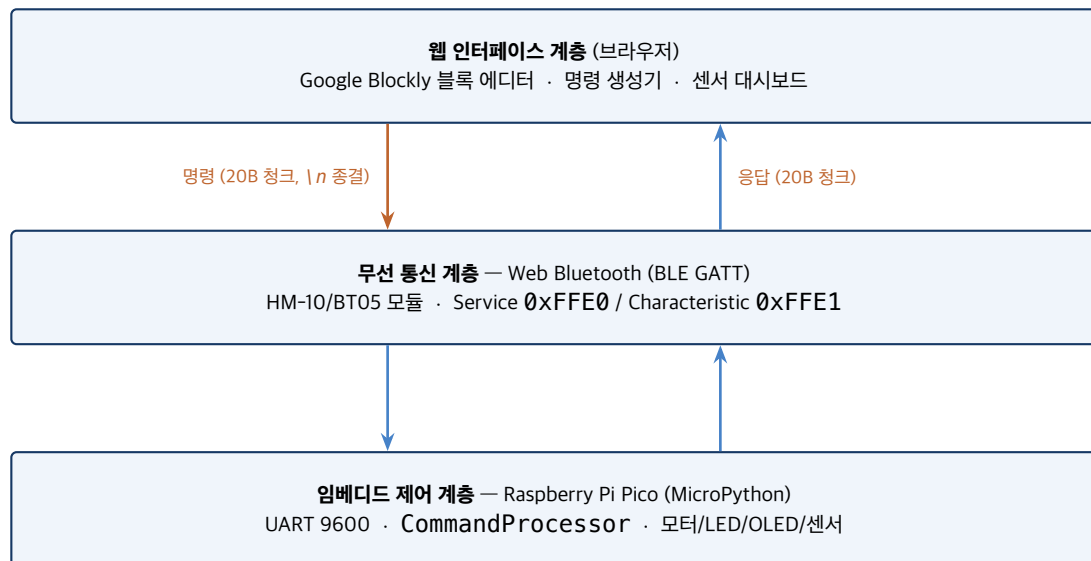


그림 1: ARES의 3계층 파이프라인과 양방향 데이터 흐름. 하향(카퍼)은 제어 명령, 상향(블루)은 센서/상태 응답이다.

데이터 흐름은 다음과 같다. 학습자가 웹 UI에서 블록을 조립하면 명령 생성기가 이를 개행으로 구분되는 텍스트 명령 문자열로 변환한다. 무선 통신 계층은 이 문자열을 20바이트 이하의 청크로 분할하여 BLE GATT characteristic에 순차적으로 기록한다. 임베디드 제어 계층은 UART를 통해 청크를 수신·재조립하여 완전한 명령으로 복원한 뒤, 접두사를 분석하여 해당 하드웨어 구동 객체로 분배한다. 센서 측정 값이나 상태 정보는 동일한 20바이트 청크 규칙을 따라 역방향으로 전송되어 웹 UI에 시각화된다.

4 핵심 기술 구현: 설계 과제와 해결

ARES 구현의 본질적 난점은 두 가지 척박한 토대 위에서 다양한 길이의 명령과 센서 응답을 양방향으로 신뢰성 있게 주고받아야 한다는 데 있다. 첫째 토대는 단가 제약상 채택하게 되는 HM-10/BT05 클론 BLE 모듈이다. 표준 사양과 달리 한 번에 보낼 수 있는 페이로드가 작고(약 20바이트), connection interval(약 30 - 70 ms)과 지원 write 모드가 제품마다 다르다. 둘째 토대는 RAM이 킬로바이트 단위인 MicroPython 런타임으로, 넉넉한 수신 버퍼나 정규식 기반 파서를 두기 어렵다. 본 장은 이 문제를 네 개의 설계 과제(C1 - C4)로 나누고, 각 과제에서 순진한 접근이 왜 실패하는지와 채택한 해법·근거를 기술한다. 표 2에 이를 요약하였다.

4.1 과제 C1·C2: 불안정한 저가 BLE 링크 위의 가변 길이 스트리밍

무선 통신 계층은 Web Bluetooth API 위에 구현되었다. 사용자가 UI에서 연결을 요청하면 브라우저의 기기 선택 절차가 활성화되며, HM-10/BT05 계열 모듈이 노출하는 UART 호환 Service UUID(0000ffe0-...)로 대상을 필터

표 2: ARES의 네 가지 설계 과제와 해결 요약

과제	순진한 접근과 그 한계	채택한 해법	효과 / 근거
C1 20B 제약 + 가변 길이 명령	대형 페이로드 일괄 전송(MTU 협상)은 클론 모듈에서 비표준 · 비호환; 고정 시작/종료 플래그(S/E)는 프레임마다 오버헤드와 escape 처리 필요	명령 끝에 개행(\n)을 프레임 종결자로 부가하고 20바이트 청크로 분할 송신	OLED 메시지 · SYS_SET · BATCH 등 장명령 정상 전송, 프레임 오버헤드 1바이트
C2 클론 모듈 비결정성	단일 write 모드 · 고정 무지연 송신은 청크 손실 또는 처리량 저하 유발	writeWithoutResponse 우선 + 미지원 시 writeWithResponse로 fallback, 청크 간 100 ms 페이싱 (마지막 청크는 생략)	interval 편차를 흡수해 청크 손실 저감, 다양한 클론 characteristic 호환
C3 비동기 응 답 정합성	송신 완료 후 응답 리스너 등록 시, 펌웨어가 빨라 직후 도착한 응답을 유실(race); 모든 명령에 응답을 받으면 응답 슬롯이 밀려 매칭 어긋남	응답 대기 명령은 송신 이전에 프로미스(타임아웃 5초) 선설정; 즉시완료 출력 명령군은 fire-and-forget으로 응답 왕복 제거	응답 정합성 확보, 매칭 어긋남 원천 차단, 처리량 향상
C4 분할 재조 립 + 자원 제약	도착분 즉시 파싱은 잘린 명령(예: BATCH)의 앞부분만 실행; 정규식 · 파싱 테이블은 RAM · 로딩 부담; 매 수신 고정 대기는 단일 명령에도 지연 부과	개행 경계까지 누적(발견 즉시 종료 / 미발견 시 500 ms 폴링), 타임아웃 시 부분 데이터 미반환, 512바이트 상한, 접두사 (startswith) 분기 디스패치	단일 명령 저지연 · 다중 명령 무결 · 저메모리 동시 충족

링하여 GATT 서버 연결을 수립하고, 데이터 송수신용 characteristic(0000ffe1-...)을 확보한다. 보안 컨텍스트 제약에 따라 플랫폼은 localhost 또는 HTTPS에서 동작한다.

문제(C1). BLE 링크에서 한 번에 전송 가능한 페이로드는 작다. ARES는 안전한 상호운용을 위해 페이로드를 **20 바이트**로 제한한다. 그런데 제어 명령은 길이가 가변적이다. 예컨대 시스템 설정 명령 SYS_SET, 80, 10, 1, ARES-Rover는 개행을 포함해 약 28바이트로 단일 청크를 초과하며, OLED 메시지 출력이나 다중 명령 일괄 전송(BATCH)은 수십~수백 바이트에 이른다. 따라서 가변 길이 명령을 작은 청크로 안전하게 분할 · 복원하는 프레임링 규칙이 필요하다.

설계 대안과 선택. 프레임링 방식의 후보와 그 한계를 표 3에 비교하였다. 고정 길이 패딩은 짧은 명령에도 최대 길이를 전송해 대역을 낭비하고, 시작/종료 플래그(S/E) 방식은 프레임마다 오버헤드가 붙을 뿐 아니라 페이로드 안에 플래그 바이트가 나타날 때 escape 처리가 필요하다. MTU 협상으로 큰 페이로드를 한 번에 보내는 방식은 표준 모듈에서는 유효하나 클론 모듈에서 협상이 비표준적이거나 실패한다. ARES는 제어 명령이 본래 텍스트라는 점을 활용하여, 명령 끝에 개행(\n)을 프레임 종결자로 부가하는 방식을 채택하였다. 오버헤드가 1바이트에 불과하고, 텍스트 명령에 자연스러운 경계이며, 수신 측의 재조립 로직(4.3절)과도 잘 맞는다.

표 3: 프레임 경계 설정 방식의 설계 대안 비교

방식	특징 / 비용	채택
고정 길이 패딩	짧은 명령도 최대 길이 전송 → 대역 낭비	
S/E 플래그 프레임링	프레임마다 오버헤드 + 페이로드 내 플래그 escape 필요	
MTU 협상(일괄 전송)	클론 모듈에서 협상 비표준 · 비호환	
개행 종결 + 20B 청크	오버헤드 1바이트, 텍스트 명령에 자연스러운 경계	✓

문제(C2)와 해법. 클론 모듈은 connection interval과 지원 write 모드가 제품마다 달라, 단일한 송신 전략으로는 안정성과 처리량을 동시에 얻기 어렵다. GATT 응답을 매 청크마다 기다리는 writeValueWithResponse만 사용하면 interval마다 ACK 왕복이 끼어들어 다중 청크 명령의 처리량이 급락하고, 반대로 지연 없이 연속 송신하면 청크가 손실된다. ARES는 응답을 기다리지 않는 writeValueWithoutResponse를 우선 사용하여 처리량을 높이되, 해당 모드를 지원하지 않는 characteristic에 대해서는 writeValueWithResponse로 안전하게 대체(fallback)한다. 또한 청크 사이에는 모듈의 connection interval(약 30 - 70 ms)을 넉넉히 넘어서는 100 ms의 페이싱 지연을 두어 청크 손실을 줄이되, 마지막 청크 직후에는 지연을 두지 않아 단일 청크 명령이 불필요하게 100 ms를 손해 보지 않게 한다(코드 1).

```
const encodedData = encoder.encode(data + '\n'); // newline-terminated frame
for (let i = 0; i < encodedData.length; i += MAX_CHUNK_SIZE) { // 20
  const chunk = encodedData.slice(i, Math.min(i + MAX_CHUNK_SIZE, encodedData.length));
  if (useWithoutResponse) await characteristic.writeValueWithoutResponse(chunk);
  else await characteristic.writeValueWithResponse(chunk);
  if (i + MAX_CHUNK_SIZE < encodedData.length) await delay(CHUNK_DELAY); // 100ms, last
    chunk skipped
}
```

Listing 1: 가변 길이 명령의 개행 종결 20바이트 청크 송신 (Web 측, 발체)

4.2 과제 C3: 비동기 양방향 통신의 정상성 확보

문제. 센서 측정처럼 응답이 필요한 명령에서, 전형적인 “송신 완료 후 응답 리스너 등록” 순서는 미묘한 경합(race)을 일으킨다. 펌웨어의 BATCH 처리가 매우 빨라, 마지막 청크의 writeValueWithoutResponse가 반환된 직후 응답 notify가 도착하는데, 이 시점에 아직 대기 핸들러가 설정되지 않았다면 응답이 처리되지 못하고 버려진다. 또한 LED 점멸이나 모터 구동처럼 즉시 완료되는 출력 명령에까지 응답 왕복을 적용하면, 그 응답이 뒤이어 보낸 다른 명령의 응답 슬롯으로 밀려 들어가 매칭이 어긋난다.

해법. 첫째, 응답 대기 명령은 송신을 시작하기 이전에 resolve · reject · 타임아웃(기본 5 초)을 담은 프로미스를 미리 설정한다. 마지막 청크 송신 직후 도착하는 응답도 이미 준비된 핸들러가 받으므로 경합이 사라지고, 무선 환경에서의 응답 손실은 타임아웃으로 회수한다. 둘째, LED · 모터 · 메시지 출력 등 즉시 완료되는 출력 명령군은 응답 왕복을 생략하는 fire-and-forget으로 처리하여(웹과 펌웨어 양측이 동일한 명령 집합을 공유) 매칭 어긋남을 원천 차단하고 처리량을 높인다. 다수 명령을 한 번에 보내야 할 때에는 BATCH;cmd1|cmd2|… 형식으로 묶어 단일 프레임으로 스트리밍하고 응답은 한 번만 회신한다.

4.3 과제 C4: 제한 자원 위의 청크 재조립과 명령 디스패치

로버의 메인 컨트롤러로는 가성비와 처리 성능이 우수한 Raspberry Pi Pico를 채택하였으며, 펌웨어는 MicroPython 환경에서 구동된다. BLE 모듈을 거쳐 유입된 데이터는 Pico의 하드웨어 UART(9600 baud) 수신 버퍼에 적재된다.

문제. 명령은 여러 청크로 쪼개져 시차를 두고 도착한다. 도착한 데이터를 즉시 완전한 명령으로 간주하면, 분할되어 절반만 도착한 BATCH 명령의 앞부분만 실행되는 오작동이 발생한다 (실제로 초기 구현에서 관찰된 결함이다). 그렇다고 매 수신마다 고정 시간을 대기하면 단일 청크 명령조차 매번 지연을 떠안는다. 또한 RAM이 제한된 MicroPython에서 큰 버퍼나 정규식 기반 파서를 두기 어렵다.

해법. 수신 측은 개행 종결 규칙에 맞추어 청크를 재조립한다. 메인 루프의 _read_uart_line은 수신 버퍼에 데이터를 누적하다가 개행 문자를 발견하는 즉시 한 줄을 완성하여 반환한다. 이로써 단일 청크 명령은 불필요한 대기 없이 즉시 처리되고, 다중 청크로 분할되어 도착하는 명령은 수신 타임아웃(500 ms)까지 폴링하며 개행까지 누적된다. 타임아웃이 발생해도 부분 데이터를 반환하지 않고 버퍼를 유지하여 다음 호출에서 이어받게 함으로써,

잘린 명령을 정상 명령으로 오인하는 문제를 제거한다. 메모리 보호를 위해 수신 버퍼 상한을 512바이트로 두고 초과 시 초기화한다.

재조립된 명령은 `CommandProcessor.process()`가 **접두사**를 분석하여 해당 하드웨어 구동 객체로 분배한다 (코드 2). 접두사 기반 분기는 정규식이나 별도 파싱 테이블 없이 결정적이고 빠르게 동작하여 제한 자원 환경에 적합하다.

```
def process(self, data):
    if data == "PING": return self._handle_ping()
    elif data == "DISTANCE": return f"DIST:{robot.distance_sensor.get_distance()}"
    elif data == "MAGNET": return f"MAG:{robot.mag_sensor.detect()}"
    elif data.startswith("SERVO_"): ... # servo wheel: direction / timed
    elif data.startswith("DC_"): ... # DC motor: PWM speed / timed
    elif data.startswith("LED_"): ... # 5-LED brightness
    elif data.startswith("MSG,"): ... # SSD1306 OLED message
    elif data.startswith("BATCH;"): ... # batched commands
    return 0
```

Listing 2: 정규식 없는 접두사 기반 명령 디스패치 (Pico 측, 발체)

응답 또한 동일한 20바이트 청킹 규칙(`_send_response`)으로 UART에 기록되어 웹 UI로 회신된다. 센서 측정 명령 DISTANCE와 MAGNET은 각각 `DIST:value`, `MAG:value` 형식의 응답을 반환한다. 주요 명령 프로토콜을 표 4에 정리하였다.

표 4: ARES 주요 명령 프로토콜(발체)

분류	명령 형식	동작
연결/시스템	PING, GET_STATUS, GET_SYS, GET_MODULES, SYS_SET, ...	연결 확인, 센서/시스템 상태 조회, 설정 저장
주행(서보)	SERVO_FORWARD, SERVO_tFORWARD,sec	연속/시간 제한 방향 이동(전후좌우), 정지
주행(DC)	DC_FORWARD, DC_tFORWARD,sec	DC 모터 정/역회전, PWM 속도 제어, 정지
LED	LED_ON,num,bri, LED_OFF,num,[v0...v4]	5구 LED 개별/패턴 밝기 제어(0.0 - 1.0)
표시/소리	MSG,text, BUZZER_ON,hz,sec	SSD1306 OLED 메시지 출력, 부저 발음
센서	DISTANCE, MAGNET	초음파 거리(→DIST:), 자기 감지(→MAG:)
일괄	BATCH;cmd1 cmd2 ...	파이프 구분 다중 명령을 단일 프레임으로 순차 실행

4.4 하드웨어 모듈 구성

하드웨어는 `hardware.py`의 단일 `robot` 싱글톤을 통해 일관되게 접근되며, 각 모듈은 `system.json` 설정으로 개별 활성화/비활성화할 수 있다. 모듈 초기화는 try-except로 보호되어, 특정 부품이 장착되지 않은 경우 해당 모듈을 `None`으로 두고 명령 처리 시 안전하게 무시한다. 이 패턴은 학습 차시별로 필요한 부품만 장착하는 교육 환경에 적합하다. 구성 모듈은 다음과 같다.

- **서보 휠(wheel):** 방향 전환 주행. 좌/우 캘리브레이션 계수를 지원한다.
- **DC 모터(dcmotor):** PWM 듀티 기반 속도 제어로 정/역회전.
- **5구 LED 배열(leds):** 채널별 PWM 듀티(duty_u16)로 0.0 - 1.0 밝기 제어, 패턴/스 와이프 효과.
- **SSD1306 OLED(oled):** 16자 자동 줄바꿈 메시지 출력.

- **초음파 거리 센서(distance, HC-SR04 계열):** 전방 장애물 거리(cm) 측정.
- **자기 센서(magsensor):** 자석 감지 여부 반환.
- **부저(buzzer):** 주파수 · 지속시간 기반 발음, 멜로디.

4.5 화성 탐사 시나리오 기반 교육과정

ARES의 학습 콘텐츠는 “화성 탐사”라는 단일 서사 아래 **12차시 × 4미션 = 48미션**으로 구조화되어 플랫폼에 내장된다. 학습자는 로버 “알비”와 함께 미션을 수행하며, 차시가 진행될수록 새로운 하드웨어 부품과 컴퓨팅 개념이 점진적으로 도입된다(표 5). 1-8차시는 LED · 부저 · DC 모터 등 개별 부품의 제어와 기초 개념(디지털 출력, 반복, 난수, PWM)을, 9차시는 1분기 회고와 발사대 제작을, 10-12차시는 5구 LED 시퀀스와 빛 · 소리 동기화를 거쳐 최종적으로 LED · 부저 · DC 모터를 통합한 “화성 로켓 발사” 시나리오와 자유 창작 발표로 마무리된다.

표 5: 화성 탐사 시나리오 12차시 구성(요약)

차시	주제	핵심 개념/부품
1	코딩 입문과 알비 만남	순차/반복, 페어링
2	LED 기초: 첫 호흡	디지털 출력, sleep
3	LED 2개로 표정 만들기	다채널 동시 제어
4	부저로 소리 만들기	주파수 × 지속시간
5	LED 3개 신호등	시퀀스 사고, 모드 분기
6	가위바위보 게임	난수, 비결정적 코드
7	DC모터 입문: 롤렛	정/역회전, PWM
8	알비 카트 주행	전후진, 가감속
9	발사대 제작/회고	1분기 총정리
10	LED 5개 카운트다운	시퀀스, 모듈화
11	LED와 부저 동기화	빛/소리 동기, 음계
12	화성 로켓 최종 발사	통합 시나리오, 창작

5 검증 및 평가

본 장은 ARES의 (1) 기능 검증, (2) 4장에서 제시한 설계 해법의 효과를 정량 검증하기 위한 무선 통신 신뢰성 측정 프로토콜, (3) 무설치 접근성의 구조적 이점을 다룬다.

※ 본 장의 정량 지표(표 6)는 측정 프로토콜과 결과 기입 양식만을 제시하며, 실제 수치는 측정 후 기입한다. 본 논문 작성 시점의 구현 범위에서 검증된 사항과 추후 측정이 필요한 사항을 구분하여 기술한다.

5.1 기능 검증

다음 항목은 구현된 플랫폼에서 동작이 확인된 사항이며, 괄호 안은 해당 동작이 검증하는 설계 과제이다.

- 중계 프로그램 설치 없이 크로미움 계열 브라우저(localhost/HTTPS)에서 HM-10/BT05 클론 모듈과의 GATT 연결 및 양방향 통신 수립.
- 20바이트 청크 스트리밍을 통한 단일/다중 청크 명령 전송과 펌웨어 측 재조립. OLED 메시지, LED 패턴, SYS_SET, BATCH 등 20바이트를 초과하는 명령의 정상 처리(C1 · C4).
- 센서 측정 등 응답 대기 명령에서 응답 경합 없이 값 회신, 출력 명령의 fire-and-forget 처리(C3).
- 표 4의 명령 집합에 대한 하드웨어 구동(서보/DC 모터, 5구 LED, OLED, 부저)과 센서 응답(DISTANCE→DIST:, MAGNET→MAG:).
- 12차시 48미션 교육과정의 블록 코딩 → 명령 생성 → 로버 구동 경로 동작.

5.2 무선 통신 신뢰성 측정 프로토콜

무선 통신의 정량적 신뢰성은 다음 프로토콜로 측정한다. 이 측정은 과제 C1 · C2(불안정 클론 링크 위의 청킹 신뢰성)와 C3 · C4(저지연 · 무결 재조립)의 해결 효과를 정량적으로 뒷받침하기 위한 것이다. 표 6는 결과 기입 양식이다.

제어 지연(latency) 웹 UI에서 명령 송신 직전 타임스탬프 t_0 를 기록하고, 해당 명령의 응답 수신 시점 t_1 을 기록하여 왕복 지연 $\Delta = t_1 - t_0$ 를 산출한다. 단일 청크 명령과 다중 청크 명령을 구분하여 각각 N 회($N \geq 100$) 반복, 평균 · 표준편차 · 95% 백분위수를 보고한다. (C2의 페이싱 · C4의 즉시 종료 효과 확인)

데이터 유실률 송신 명령에 일련번호를 부여하고 펌웨어가 에코하도록 하여, 연속 송신 M 건 중 응답 누락 · 불일치 건수를 집계한다. 유실률 = (누락 + 불일치)/ M . (C1의 프레임링 · C3의 정합성 효과 확인)

측정 환경 브라우저 · OS · BLE 모듈 펌웨어 버전, 송수신 거리, CHUNK_DELAY 설정값을 함께 기록한다.

표 6: 무선 통신 신뢰성 측정 결과 양식(측정 후 기입)

지표	단일 청크	다중 청크	비고
평균 왕복 지연 (ms)	—	—	$N=?$
지연 표준편차 (ms)	—	—	
95% 백분위 지연 (ms)	—	—	
데이터 유실률 (%)	—	—	$M=?$

5.3 무설치 접근성

ARES는 설계상 어떠한 중계 프로그램 · 드라이버 · 확장도 설치하지 않으며, 브라우저로 페이지에 접속하는 것만으로 즉시 사용 가능하다. 이는 애플리케이션 · connector 설치와 권한 설정이 단말마다 필요한 종래 방식과 대비되는 구조적 이점이다. 관리형 단말(크롬북 · 웨일북) 환경에서 임의 설치가 제한되더라도 도입이 가능하다는 점에서 교육 현장 적용성이 높다. 다만 설치 시간 단축 효과의 정량화(교실 단위 준비 시간 비교)는 통제된 현장 연구를 통한 후속 측정이 필요하다.

6 결론 및 향후 과제

본 연구는 Web Bluetooth 웹 표준 기술을 기반으로, 중계 프로그램 설치 없이 브라우저만으로 교육용 로버를 직접 제어하는 무설치 플랫폼 ARES를 설계 · 구현하였다. 본 연구의 실질적 기여는 사양이 제각각인 저가 클론 BLE 모듈과 자원이 제한된 MicroPython 런타임이라는 비이상적 토대 위에서 다양한 길이의 명령과 센서 응답을 신뢰성 있게 주고받는 문제를 네 가지 설계 과제로 정식화하고 해결한 데 있다. 즉, BLE 20바이트 제약을 극복하는 개행 종결 청크 스트리밍(C1), 클론 모듈의 비결정성을 흡수하는 전송 모드 선택과 페이싱(C2), 비동기 응답 경합을 제거하는 프로미스 선선평과 fire-and-forget 분리(C3), 그리고 제한 자원 위에서의 안전한 청크 재조립과 접두사 기반 디스패치(C4)를 구현하였다. 또한 화성 탐사 서사를 중심으로 한 12차시 48미션 교육과정을 플랫폼에 내장하여, 경량 운영체제 기반 단말이 보급된 교육 현장의 설치 장벽 문제를 구조적으로 해소하였다.

향후 과제는 다음과 같다. 첫째, Web Bluetooth를 지원하지 않는 브라우저(예: iOS Safari) 환경에서도 하드웨어 제어가 가능하도록 WebSocket 브리지 등 대체 통신 경로를 연구한다. 둘째, 다수의 로버가 협력 미션을 수행하는 멀티 에이전트 화성 탐사 시나리오와 이를 위한 통신 스케줄링 · 최적화 알고리즘을 연구한다. 셋째, 5장에서 제시한 측정 프로토콜에 따라 통신 신뢰성을 정량 측정하여 네 가지 설계 해법의 효과를 입증하고, 통제된 현장 연구를 통해 설치 시간 단축과 학습 효과를 검증한다.

참고문헌

[1] M. Resnick et al., "Scratch: Programming for All," Communications of the ACM, vol. 52, no. 11, pp. 60-67, 2009.

- [2] 네이버 커넥트재단, “엔트리(Entry) 블록 코딩 플랫폼,” <https://playentry.org>.
- [3] N. Fraser, “Ten Things We’ve Learned from Blockly,” IEEE Blocks and Beyond Workshop, pp. 49-50, 2015.
- [4] Web Bluetooth Community Group, “Web Bluetooth,” W3C Community Group Draft Report. <https://webbluetoothcg.github.io/web-bluetooth/>.
- [5] D. George et al., “MicroPython,” <https://micropython.org>.
- [6] Raspberry Pi Ltd., “Raspberry Pi Pico Datasheet,” 2021.